

Good

Evening



Everyone :-

DISADVANTAGES OF ARRAYS

- ✓ you can't increase / decrease size
- ✓ Size should be known in advance.

ArrayList

- ✓ Dynamic Size
- ✓ no need to know size in advance

Example -

- New tabs on browser
- ↳ Youtube Playlist
- ↳ Shopping list

Syntax:-

`ArrayList < Type > arr = new ArrayList < Type > ();`

Type ⇒ int, float, double, char boolean
 ↓ ↓ ↓
 Integer Float Double Character Boolean.

{ }

`ArrayList < Integer > arr = new ArrayList < Integer > ();`

Basic Operations

✓ Inserting element

• add .

{ 10, 12 }

arr.add (10);
arr.add (12);

✓ Get

arr.get (0)

⇒ 10

✓ Size

arr.size();

✓ Remove

arr.remove (1);

{ 10, }

✓ Set

arr.set (0, 61)

{ 61 }



Taking ArrayList as an Input

```
public static void main ( ) {
```

done on ide.

Given an arraylist as input, return Arraylist of the multiples of 5 and 7.

Input: [5 , 7 , 70 , 50 , 35]

O/P : [70 , 35]

Code:

```
ArrayList<Integer> multiples (ArrayList<Integer> arr) {
```

```
    ArrayList<Integer> ans = new ArrayList<Integer>();
```

```
    for (int i=0; i<arr.size(); i++) {
```

```
        int val = arr.get(i);
```

```
        if (val%2 == 0 && val%7 == 0) {
```

```
            ans.add(val);
```

```
        }
```

```
    }
```

```
    return ans;
```

```
}
```

Given two integers A and B as input, return an Arraylist containing first B multiples of A.

Input : $A = 2$ $B = 4$

Output: $2 \times 1 = 2$
 $2 \times 2 = 4$
 $2 \times 3 = 6$
 $2 \times 4 = 8$.

Code.

```
ArrayList <Integer> firstB (int A, int B){
```

done on side.

```
}
```

2D ArrayList

2D - Array of arrays.

2D ArrayList $\rightarrow$ ArrayList of ArrayLists

Syntax: -

```
ArrayList<ArrayList<Type>> mat = new ArrayList<ArrayList<Type>>();
```

```
ArrayList<ArrayList<Integer>> mat = new ArrayList<ArrayList<Integer>>();
```

, { }

```
ArrayList<Integer> list1 = new ArrayList<Integer>();
```

```
list1.add(10);
```

```
list1.add(11);
```

```
list1.add(12);
```

```
list1 = {10, 11, 12}
```

```
ArrayList<Integer> list2 = new ArrayList<Integer>();
```

```
list2.add(20);
```

```
list2.add(21);
```

```
list2.add(22);
```

```
list2 = {20, 21, 22}
```

```
mat.add (list1);  
mat.add (list2);
```

mat $\Rightarrow$ {

{ 10, 11, 12 }

{ 20, 21, 22 }

}

```
ArrayList <Integer> l3 = new ArrayList <Integer> ();
```

```
l3.add (1);
```

mat $\Rightarrow$ {

▷ { 10, 11, 12 }

└ { 20, 21, 22 }

✱ { 1 }

}

```
mat.get(1).set(2, 25);
```

mat.get(2) $\Rightarrow$ (1)

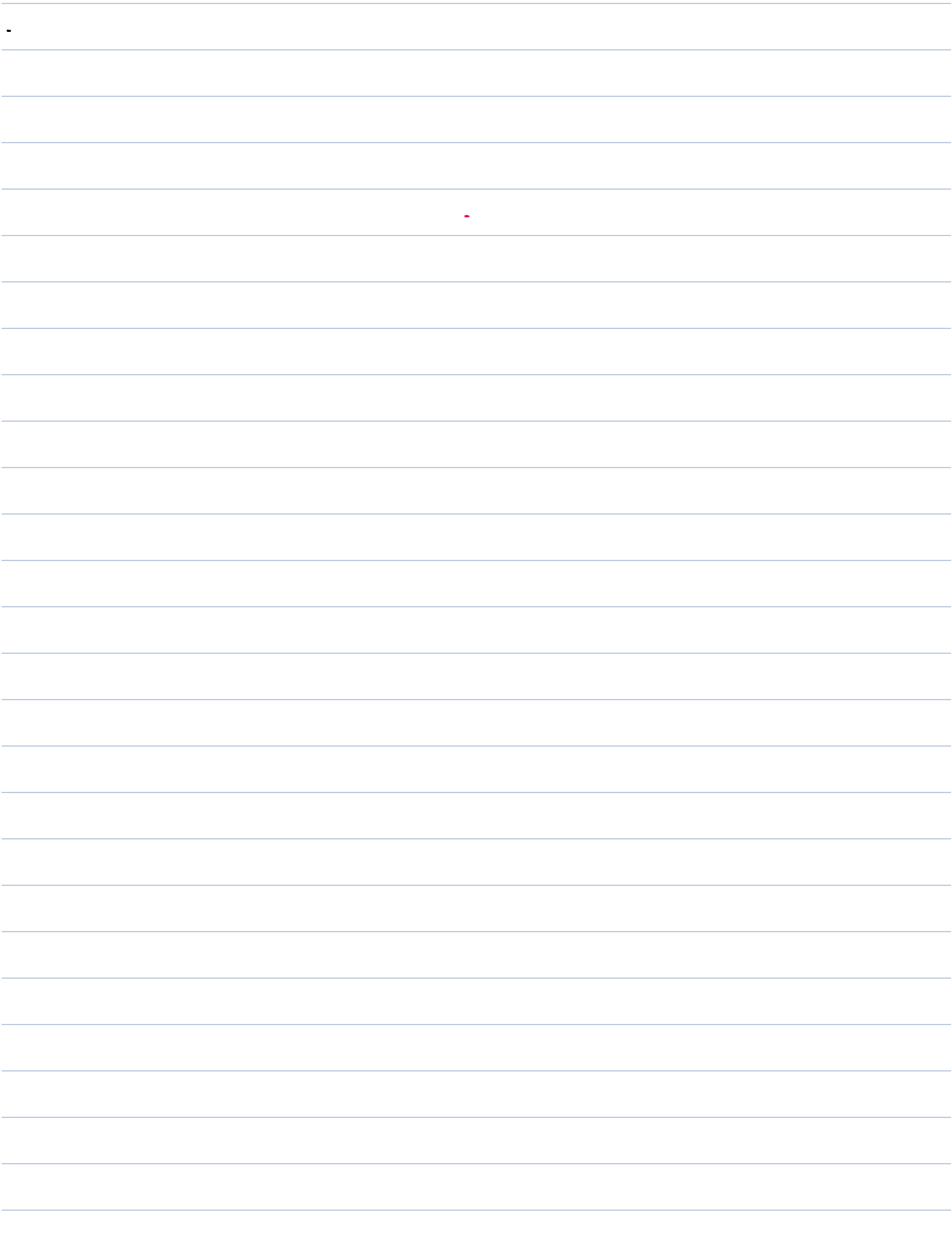
mat.size() $\Rightarrow$ 3

mat.get(2).size() $\Rightarrow$ 1

mat.get(1).get(1) $\Rightarrow$ 21

mat.get(1).get(2) $\Rightarrow$ 22

mat.get(0).get(0) $\Rightarrow$ 10



Given a 2D ArrayList as input, print it line by line

Input: `al` `[[1, 4], [0], [10, -5, 1]]`

```
ArrayList<ArrayList<Integer>> mat = new ArrayList<ArrayList<Integer>>()
```

Code:-

```
void print2DList (ArrayList <ArrayList <Integer>> list2d) {
```

```
    for (int i=0; i < mat.size(); ++i) {
```

```
        {
```

```
            for (int j=0; j < mat(i).size(); ++j) {
```

```
                System.out.print (mat.get(i).get(j) + " ")
```

```
            }
```

```
        } soln(),
```

```
    }
```

```
}
```

Given an integer N as input, Return numeric staircase

Example

$\Rightarrow 2 \Rightarrow$

[1 [1]

2 [1 2]

3 [1 2 3]

]

{

[1]

[1 2]

[1 2 3]

}

```
ArrayList <ArrayList <Integer>> staircase (int N) {
```

```
    ArrayList <ArrayList <Integer>> ans = new ArrayList <ArrayList <Integer>> ();
```

```
    for (int row = 1; row <= N; row++) {
```

```
        ArrayList <Integer> row = new ArrayList <Integer> ();
```

```
        for (int col = 1; col <= row; col++) {
```

```
            row.add (col);
```

```
        }
```

```
        ans.add (row);
```

```
    }
```

```
    return ans;
```

```
}
```

